Containers and Docker

ship the app with everything it needs

Watch the 4-min video

In one line: a container image packages your code with its runtime, libraries and start command; containers run it as isolated processes that **share the host kernel** (namespaces + cgroups), so they're light and fast; build images in cache-friendly layers with multi-stage builds, and ship them by immutable tag from a scanned registry.

1 It works on my machine

The FastAPI service runs perfectly on a developer's laptop, then fails in production: the server has Python 3.9 instead of 3.12 and is missing `libpq`. Environments drift: OS packages, runtime versions, environment variables. Containers fix this by shipping the application **together with** everything it needs, as one immutable artifact that runs the same on a laptop, a CI runner and a Kubernetes cluster.

2 Images and containers

An **image** is a packaged filesystem plus metadata: OS libraries, the language runtime, your dependencies and code, environment defaults and the start command. Under the OCI image specification it is a stack of read-only **layers** (tar archives) plus a config, each addressed by its SHA-256 digest, so identical layers are stored and downloaded once. A **container** is a running instance: the runtime stacks the image layers with a union filesystem (overlayfs), adds a thin writable layer on top, and starts the process. Anything written there disappears with the container, so state belongs in volumes or external services.

3 Containers are not virtual machines

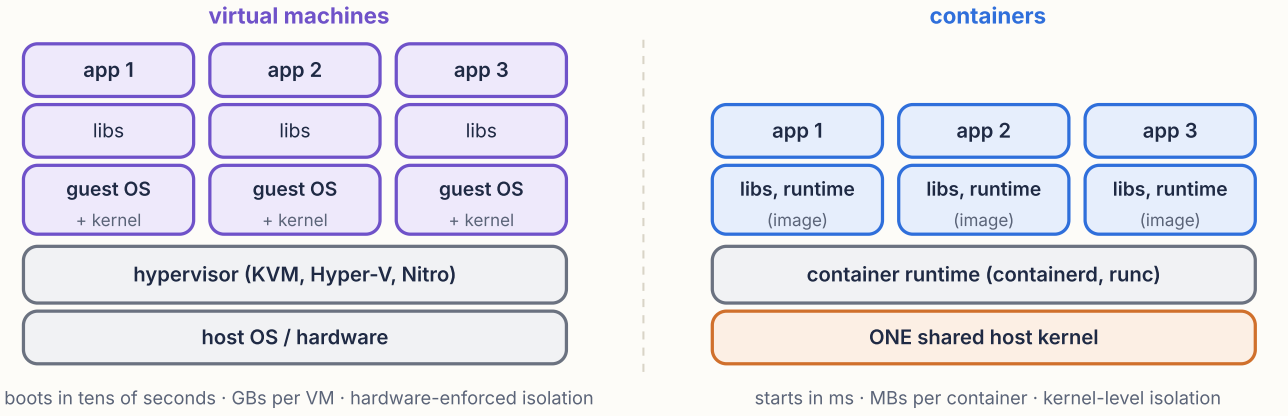


Figure 1. A VM virtualises hardware and runs a whole OS; a container is an isolated process on the host's kernel.

A container is an ordinary Linux process with two kernel features applied. **Namespaces** give it its own view of the system: PID (its own process tree, where it is PID 1), network (own interfaces and ports), mount (own filesystem), UTS (hostname), IPC and user namespaces. **Control groups (cgroups)** limit what it may consume: CPU, memory, I/O and number of processes; these are what Kubernetes requests and limits set (lesson 57).

	Virtual machine	Container
Isolation boundary	hypervisor, hardware-assisted	kernel namespaces + cgroups
Contains	full guest OS and kernel	app + user-space libraries only
Start time	tens of seconds to minutes	milliseconds to a second or two
Footprint	GBs of memory and disk	what the process needs; layers shared
Kernel	any (Windows on Linux host is fine)	the host's (Linux images need a Linux kernel)
Security	strong; a guest escape is rare	weaker: a kernel bug can break out

When tenants run untrusted code, the industry answer is to combine both: **sandboxed runtimes** such as gVisor (a user-space kernel) or micro-VMs like Firecracker and Kata Containers, which keep container ergonomics but put a VM-style boundary around each workload.

4 Layers and build caching

Each Dockerfile instruction that changes the filesystem (`RUN`, `COPY`, `ADD`) creates a layer, and the builder reuses a cached layer when the instruction and its inputs are unchanged. A changed layer invalidates every layer after it, so **order matters**: copy `requirements.txt` and install dependencies first, then copy the source. A code change then rebuilds a one-second layer instead of reinstalling every package. Add a `.dockerignore` (`.git`, `.venv`, tests, secrets) so the build context is small and unrelated file changes don't bust the cache.

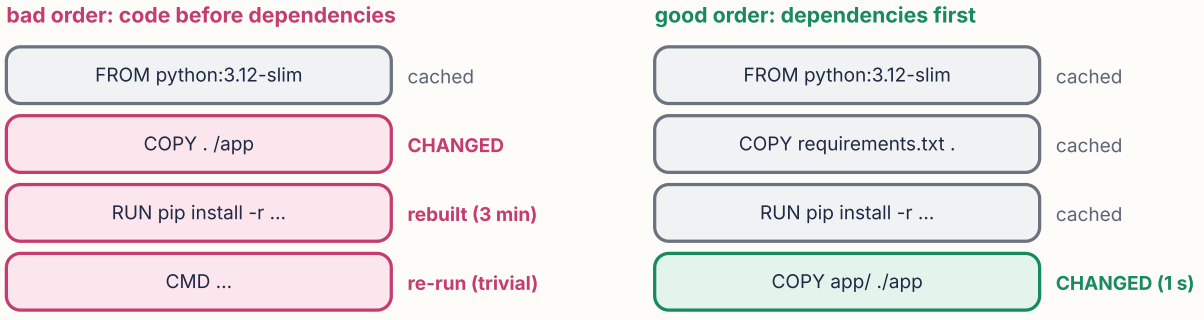


Figure 2. Same Dockerfile, two orders. The left one reinstalls every dependency on each code change.

5 Multi-stage builds and small, safe images

Build tools (compilers, headers) are needed to build but not to run. A **multi-stage build** compiles in a first stage and copies only the results into a fresh, slim final stage. Smaller images pull and start faster (which matters when autoscaling) and contain fewer packages with known vulnerabilities. Choosing a base for Python: `python:3.12-slim` (Debian) is the usual choice; Alpine uses musl libc, so many Python wheels must be compiled from source; distroless images strip even the shell and package manager.

Three details in the file below matter. A file deleted in a later layer still exists in the earlier one, so `COPY` + `RUN rm` doesn't shrink an image; the bind mount avoids the copy altogether. **Run as non-root** (`USER 10001`) so a compromised process has fewer privileges. And use the **exec form** of `CMD` (a JSON array): the shell form wraps the process in `/bin/sh -c`, which doesn't forward SIGTERM, so the app is killed hard instead of shutting down gracefully.

```
# syntax=docker/dockerfile:1
FROM python:3.12-slim AS build
RUN apt-get update && apt-get install -y --no-install-recommends build-essential
WORKDIR /src
COPY requirements.txt .
RUN pip wheel --no-cache-dir --wheel-dir /wheels -r requirements.txt

FROM python:3.12-slim                     # fresh, slim final stage
WORKDIR /app
RUN --mount=type=bind,from=build,source=/wheels,target=/wheels \
    pip install --no-cache-dir --no-index /wheels/*   # wheels never become a layer
COPY app/ ./app
RUN useradd --uid 10001 --no-create-home appuser
USER 10001
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Figure 3. Multi-stage FastAPI image. The bind mount (BuildKit, the default builder since Docker Engine 23) installs the wheels without storing them in a layer.

6 Registries, tags, scanning and signing

Images live in a **registry**: Docker Hub, Azure Container Registry, Amazon ECR, GitHub Container Registry, Google Artifact Registry. Tag each build with an **immutable version**, typically the git commit SHA, not `latest`: `latest` moves, so two nodes can run different code under the same name and rollbacks become guesswork. For full certainty deploy by digest (`shop-api@sha256:…`). In CI, **scan** images for known CVEs (Trivy, Grype, or the registry's scanner), generate an SBOM, and **sign** them (Sigstore cosign, Notation) so the cluster's admission policy only runs images your pipeline built.

WORKED EXAMPLE: A SMALL FASTAPI IMAGE

The first version was `FROM python:3.12` (the full Debian image with compilers), `COPY . .`, then `pip install`: about **1.1 GB**, rebuilt from scratch on every commit, running as root. The multi-stage version above builds wheels in stage one, installs them in a slim stage two, copies only `app/`, runs as UID 10001 and starts uvicorn in exec form: about **150 MB**. Code-only commits rebuild in seconds because the dependency layers are cached, and the CI pipeline pushes `shopreg.azurecr.io/shop-api:3f9c2e1` after a clean scan.

COMMON MISTAKES

- Copying the whole source tree before installing dependencies.
- Deploying `:latest`, so you can't tell or pin what's running.
- Running as root and baking secrets into image layers (they stay in history).
- Treating containers as a strong security boundary for untrusted code.

7 Key terms

Image: layered, immutable filesystem + start command

Container: running instance of an image

Namespaces: kernel feature: isolated view of processes, network, mounts

cgroups: kernel feature: CPU, memory, I/O limits

Multi-stage build: build in one stage, ship only results

Registry: stores and serves images by tag and digest

8 Interview questions

Q1. How are containers different from VMs? BASIC

Containers share the host kernel and isolate processes with namespaces and cgroups, so they're lightweight and start in milliseconds; VMs run a full guest OS on a hypervisor, with stronger isolation and more overhead.

Q2. How do you keep images small and fast to build? BASIC

Order layers for caching, use multi-stage builds with slim bases, copy only runtime artifacts, and use a .dockerignore.

Q3. Image vs container? BASIC

An image is the immutable, layered package; a container is a running instance of it with its own writable layer.

Q4. Every code change triggers a five-minute pip install. Why? SCENARIO

The source is copied before dependencies are installed, so any code change invalidates the install layer; copy requirements.txt and install first, then copy the code.

Q5. A pod takes 30 seconds to stop on every deploy. What's a likely cause? SCENARIO

The app doesn't receive SIGTERM (shell-form CMD wraps it in sh), so it ignores the stop signal until the grace period ends and it's killed; use exec form and handle SIGTERM.

Q6. You deleted a secret file in a later RUN step. Is it gone from the image? TRICKY

No, it remains in the earlier layer and can be extracted; use build secrets (RUN --mount=type=secret) so it never enters a layer.

Q7. Why is :latest a bad tag in production? TRICKY

It's mutable: different nodes can pull different images under the same name, and you can't reliably roll back or know what's running.

Next: SD-V-006 · Kubernetes Core Objects backendroadmap.com